Assignment 4 Report: Neural Network Verification with alpha-beta-CROWN

1. Overview

In this assignment, I explored alpha-beta-CROWN and used it to verify a neural network robustness property. The project includes an external neural network model, an ONNX model file, a VNNLIB specification, a YAML configuration file, a reproducible test script, and verification results. The main goal was to check whether a small neural network remains locally robust under a bounded input perturbation.

2. alpha-beta-CROWN Structure

The alpha-beta-CROWN project is organized around a complete verifier. The main executable used in this assignment is abcrown.py inside complete_verifier. Verification settings are controlled through YAML files. Important YAML sections include general, model, data, specification, solver, attack, and bab. The model section points to a PyTorch or ONNX model, the specification section points to the VNNLIB property, and solver/bab configure alpha-CROWN, beta-CROWN, and branch-and-bound behavior.

I first tested the provided CIFAR convolutional example to confirm that the verifier could load a model, read a configuration file, run PGD attack, and produce verification statuses such as unsafe-pgd and safe-incomplete. Since the built-in CIFAR configuration targets many samples and is expensive on CPU, I used it only as an installation and output-format check.

3. External Model and Dataset

For the external model, I used the Iris dataset from scikit-learn. The Iris dataset has four numerical input features and three output classes. I trained a small multi-layer perceptron and exported it to ONNX. The model architecture is:

```
Linear(4, 16)
ReLU
Linear(16, 16)
ReLU
Linear(16, 3)
```

The model was trained in create_iris_model.py. The final ONNX model is saved as models/iris_mlp.onnx. The selected test sample and preprocessing information are saved in data/iris_test_sample.npz. The ONNX model was checked successfully with ONNX checker and uses opset 12.

4. Verification Property

The verification property checks local robustness for one selected Iris test sample. The input is allowed to vary inside an L-infinity perturbation box with epsilon = 0.05. The VNNLIB file encodes the unsafe condition: another class output becomes greater than or equal to the originally predicted class output. Therefore, if the verifier returns UNSAT, it means that no adversarial input exists inside the perturbation region.

The unsafe condition was encoded as a disjunction over the non-predicted classes. For example, if class 0 is the predicted class, the VNNLIB property checks whether either Y_1 >= Y_0 or Y_2 >= Y_0 can occur within the input bounds. This follows the standard adversarial specification style: the verifier searches for a counterexample, and UNSAT means the robustness property holds.

5. Verification Configuration

The YAML configuration file is configs/iris_mlp.yaml. It uses CPU execution, the ONNX model path, the VNNLIB specification path, alpha-CROWN bound propagation, beta-CROWN optimization, and branch-and-bound as the complete verifier. The run can be reproduced by executing test.py or by directly running abcrown.py with the YAML configuration.

```
python test.py

cd external/alpha-beta-CROWN/complete_verifier
python abcrown.py --config ../../../configs/iris_mlp.yaml
```

6. Result and Interpretation

For the selected Iris sample with epsilon = 0.05, alpha-beta-CROWN returned:

```
PGD attack failed
Total number of violation: 0
Verified with initial CROWN!
Result: unsat
```

The result UNSAT means that the unsafe adversarial condition is unsatisfiable. In other words, within the specified L-infinity perturbation box, there is no input that makes another class output greater than or equal to the predicted class output. Therefore, the model is verified to be locally robust for this sample and epsilon value. The verification finished quickly on CPU because the Iris MLP is small and only one input instance was verified.

7. Comparison with Marabou

Compared with Marabou, alpha-beta-CROWN is more specialized for neural network robustness verification using bound propagation and branch-and-bound. It supports methods such as CROWN, alpha-CROWN, beta-CROWN, PGD attack, and BaB splitting. This makes it convenient for ONNX plus VNNLIB robustness experiments. Marabou, on the other hand, provides a more explicit SMT-style view of neural network verification by reasoning over piecewise-linear constraints.

In my previous Marabou-based workflow, the verification problem felt closer to manually constructing input bounds and output constraints. In alpha-beta-CROWN, the YAML and VNNLIB format made the experiment more standardized. However, alpha-beta-CROWN was also sensitive to configuration paths and VNNLIB syntax, so debugging required careful checking of relative paths, output-condition direction, and parser-compatible specification format.

8. Strengths and Limitations

The main strength of alpha-beta-CROWN is that it can combine fast incomplete methods, adversarial search, and complete branch-and-bound verification. In this experiment, PGD failed to find a violation, and the initial CROWN bound was already strong enough to prove the property. This shows that alpha-beta-CROWN can efficiently certify local robustness for small neural networks.

The main limitation is that verification can become expensive for larger models, larger input dimensions, or larger perturbation radii. The built-in CIFAR example was much slower on CPU, especially when many samples were selected. Another limitation is that a single verified sample does not prove global robustness of the entire model. It only proves local robustness for the selected input and epsilon. For stronger evidence, more samples and multiple epsilon values should be verified.

9. Conclusion

This assignment successfully verified an external Iris MLP model using alpha-beta-CROWN. The model was exported to ONNX, the robustness property was encoded in VNNLIB, and the verification was executed through a YAML configuration and test.py. The final result was UNSAT for the unsafe condition at L-infinity epsilon = 0.05, meaning the selected sample is locally robust under the tested perturbation bound.